

V-Scale: Budget-Aware Inference Scheduling for Video Generation

Stanford CS348K — Visual Computing Systems — Spring 2026

Jenny Jin Jeffrey Liu
{yqjin, liujy3}@stanford.edu

Abstract

Modern video generation systems expose many quality-related controls, including model choice, denoising steps, frame count, resolution, and CPU offload. These controls are often treated as fixed presets, but fixed presets ignore the latency and memory budgets that determine whether a model can be served interactively. We build V-Scale, a profiling-based scheduler that compares configurations by their measured quality–latency–memory tradeoffs and selects the highest-quality feasible configuration for a given budget. Across our measured configurations, resource constraints substantially change model comparison: 49 of 54 profiled configurations become dominated once latency is considered, and V-Scale achieves 17 wins, 1 tie, and 0 losses against fixed-preset baselines.

Key Takeaways

1. **Latency changes which configurations are competitive.** Configurations that look attractive by quality alone can become non-competitive once latency is considered. In our sweep, 49 of 54 profiled configurations are dominated by another configuration with higher quality and lower latency.
2. **Denoising work is the main latency bottleneck.** Across all three sweep axes—steps, frame count, and resolution—additional denoising work is the main contributor to end-to-end latency. This cost grows much faster for CogVideoX-2B than for LTX-Video, making lower-latency LTX-Video configurations more competitive under tight budgets.

3. **Memory and latency trade off through CPU offload.** Averaged across three matched CogVideoX-2B configurations, CPU offload reduces peak GPU memory from 18.8 GB to 11.1 GB, but increases latency from 16.5 s to 22.1 s. Offload changes feasibility under memory limits, not just runtime.
4. **Budget-aware scheduling outperforms fixed presets.** By scheduling over the Pareto-optimal frontier rather than committing to one preset, V-Scale achieves 17 wins, 1 tie, and 0 losses against fixed presets across 18 budget decisions, with a mean quality gain of +0.0549.

1 Background and Setup

1.1 Problem Definition

Video generation models expose a large configuration space: model choice, denoising steps, frame count, spatial resolution, and memory-management mode. In practice, a user or serving system must choose one concrete configuration before inference begins. A common deployment strategy is to use a fixed preset: one model and one set of generation parameters chosen offline.

Fixed presets are simple, but they ignore the central systems constraint: the best configuration depends on the resource budget. A quality-oriented preset may exceed an interactive latency target, while a speed-oriented preset may leave quality unused when more time is available. Similarly, a memory-saving option such as CPU offload can make a configuration feasible on a smaller GPU, but can also increase runtime. Therefore, comparing video generation models by unconstrained

quality alone can be misleading.

1.2 Inputs, Outputs, and Constraints

Inputs. A text prompt p , a latency budget B in seconds, and a hardware target H .

Output. A selected generation configuration $c = (\text{model}, \text{steps}, \text{frames}, \text{resolution}, \text{offload})$.

Constraints. The selected configuration must satisfy $\text{latency}(c, H) \leq B$. When memory is constrained, it must also fit within the available GPU memory.

Subject to these constraints, V-Scale selects the configuration with the highest measured quality:

$$c^* = \arg \max_{c \in \mathcal{C}} \text{quality}(c) \quad \text{s.t.} \quad \text{latency}(c, H) \leq B.$$

1.3 Research Question and Hypothesis

Our research question is:

How do latency and memory constraints change the comparison of video generation models and configurations, and can a budget-aware scheduler exploit these tradeoffs to improve delivered quality over fixed presets?

Our hypothesis is that resource constraints materially change which configurations are competitive. In particular, we expect that many high-quality configurations become dominated or infeasible once latency and memory are considered. If this is true, then a scheduler that selects from the Pareto-optimal frontier should match or exceed the quality of fixed presets under the same latency budget.

1.4 Technical Challenge

The challenge is that video generation models cannot be compared by quality alone: each configuration choice changes latency and memory use on the target hardware. The hard part is characterizing the quality–latency–memory tradeoff across two different video generation systems and turning that characterization into a scheduling rule. Quality and latency are only known after inference, so the scheduler first needs a profiling phase that measures configurations on the target

hardware. The scheduler must then use these measurements to decide which configurations are actually worth considering under a resource budget.

2 Approach

2.1 Starting Point and Contributions

We built V-Scale on top of two pretrained video generation models: LTX-Video [1] and CogVideoX-2B [2], using their Hugging Face `diffusers` pipelines. We did not modify model weights or architectures. Our contribution is a systems layer around these models:

1. a profiling harness that sweeps model and generation settings;
2. a validity gate and lightweight reference-free quality proxy;
3. Pareto-frontier construction over measured latency and quality;
4. a budget-aware scheduler that selects the highest-quality feasible configuration;
5. analysis of bottlenecks, CPU offload, and fixed-preset baselines.

2.2 Models and Configuration Space

- **LTX-Video.** A fast video generation model that can produce valid outputs using relatively few denoising steps. We evaluate step counts from 4 to 30, frame counts from 17 to 81, and two resolution tiers.
- **CogVideoX-2B.** A higher-cost video generation model that typically requires more denoising steps for valid output. We evaluate 15, 30, and 50 steps, multiple frame counts, a fixed high-resolution setting, and CPU offload mode.

Across both models, the sweep covers denoising steps, frame count, resolution, and offload mode, yielding 54 profiled configurations.

2.3 Prompt Set

We use three prompts that span different motion regimes:

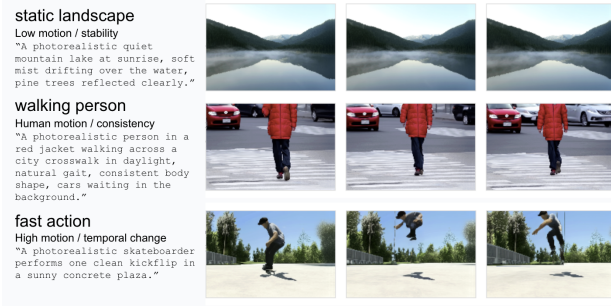


Figure 1: Representative frames from the three prompt categories used in evaluation. The prompt set spans low-motion (static landscape), medium-motion (walking person), and high-motion (fast action) regimes.

- **Static landscape:** low motion and visual stability.
- **Walking person:** human motion and spatial consistency.
- **Fast action:** rapid temporal change and higher temporal complexity.

These prompts are intentionally chosen to exercise different quality–latency tradeoffs. For example, high-motion prompts may benefit more from additional frames, while human-motion prompts may benefit more from spatial detail and consistency.

2.4 Evaluation Metrics

Latency. We measure end-to-end wall-clock latency on an NVIDIA L40S GPU, from inference start to the first readable generated video. This includes text encoding, the denoising loop, and VAE decoding.

Quality. We use a two-stage quality procedure. First, a validity gate rejects outputs that are blank, nearly static, or unstable noise. For valid outputs, we compute a reference-free proxy score that combines stability, motion, and sharpness. We use this proxy because the scheduler needs a consistent scalar score for relative configuration decisions. We manually spot-checked representative outputs to verify that the proxy rankings were reasonable for the comparisons in this project.

2.5 V-Scale Pipeline

V-Scale has four stages, shown in Figure 2.

1. **Profile:** run each configuration on the target hardware and record latency, memory, and quality proxy.
2. **Prune:** construct the Pareto frontier by removing configurations that are both slower and lower quality than another configuration.
3. **Schedule:** for a given latency budget, filter to feasible configurations and select the highest-quality remaining option.
4. **Serve:** run the selected configuration.

Profiling is performed offline for a hardware target. At request time, scheduling is a lookup over the measured frontier, not a full online search.

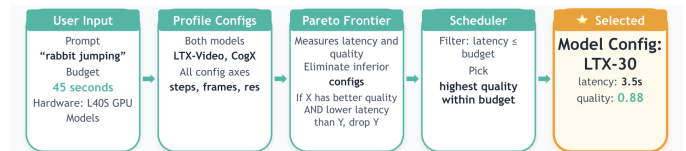


Figure 2: The V-Scale pipeline. Profiling characterizes the configuration space once per hardware target; scheduling selects the highest-quality feasible configuration for each latency budget.

2.6 Iteration and Design Choices

Our first quality metric was CLIP similarity between the prompt and generated frames. This did not reliably capture visible video quality in our sweep: some low-step runs could score reasonably on prompt similarity while producing blurry, static, or temporally incoherent video. We therefore replaced it with the validity gate and composite proxy described above.

We also tested aggressive step reduction for CogVideoX-2B to make it more latency-competitive with LTX-Video. This did not work well: very low-step CogVideoX-2B runs often failed the validity gate or produced much lower proxy scores than LTX-Video at similar latency. This motivated our focus on measured Pareto tradeoffs rather than assuming the same parameter changes have the same effect across models.

3 Evaluation and Results

3.1 Definition of Success

Our goal is to determine whether latency and memory constraints change which video generation configurations are competitive, and whether a budget-aware scheduler can use those constraints to improve delivered quality over fixed presets. We consider the project successful if two conditions hold. First, the profiling experiments should identify measurable quality–latency–memory tradeoffs that change the set of competitive configurations. Second, under the same latency budgets, V-Scale should match or exceed the quality of fixed presets.

We evaluate these criteria using controlled latency sweeps, a CPU-offload ablation, Pareto-frontier analysis, and a fixed-preset comparison. The main system-level metric is win/tie/loss count against fixed presets, with mean quality gain as the secondary metric.

3.2 Experimental Setup

All experiments are run on a single NVIDIA L40S GPU. The dataset consists of three text prompts chosen to cover low-motion, medium-motion, and high-motion video generation regimes. We profile 54 configurations across LTX-Video and CogVideoX-2B, varying denoising steps, frame count, resolution, and CPU offload. For each generated video, we record validity, quality proxy score, latency, and peak GPU memory where relevant.

Our baseline is a fixed-preset policy: predefined LTX-Video and CogVideoX-2B preview/quality configurations are selected without adapting to the prompt or latency budget. A fixed preset is included in a comparison only if it produces a valid output and satisfies the same latency budget as V-Scale. We evaluate 24 prompt-budget scheduling decisions, of which 18 have at least one comparable fixed-preset baseline.

3.3 Controlled Sweeps: What Drives Latency?

We first isolate runtime behavior by sweeping one axis at a time: denoising steps, frame count, and

resolution. Table 1 shows the latency increase from the low to high setting for each sweep.

Table 1: Latency sensitivity to configuration changes, averaged across prompts. CogVideoX-2B exhibits substantially larger latency increases than LTX-Video across all sweep axes.

Sweep	CogX	LTX
Steps	+60 s	+2.6 s
Frames	+57 s	+2.9 s
Resolution	+12 s	+1.4 s

Table 1 quantifies the marginal latency cost of increasing each quality-related setting. The key pattern is not just that larger settings are slower; it is that the cost is highly model-dependent. Increasing denoising steps adds about 60s for CogVideoX-2B but only 2.6s for LTX-Video, a roughly $23\times$ larger increase. Increasing frame count shows a similar gap: 57s for CogVideoX-2B versus 2.9s for LTX-Video. Resolution has a smaller absolute effect, but still increases latency much more for CogVideoX-2B.

This means the scheduler cannot treat steps, frames, or resolution as generic hyperparameters with model-independent cost. The same configuration change can be cheap for one model and prohibitively expensive for another, which is why model comparison must be tied to measured latency rather than quality settings alone.

3.4 Bottleneck Analysis: Denoising Work

The common mechanism behind all three sweeps is denoising work. More steps mean more denoising iterations, more frames mean more frames to denoise, and higher resolution means more pixels per denoising step. This explains why the three plots in Figure 3 have the same qualitative shape.

This explains the model gap. LTX-Video typically uses far fewer denoising steps than CogVideoX-2B, so its latency grows more slowly as quality-related settings scale. CogVideoX-2B pays a higher marginal cost because increasing frames or resolution multiplies an already expensive denois-

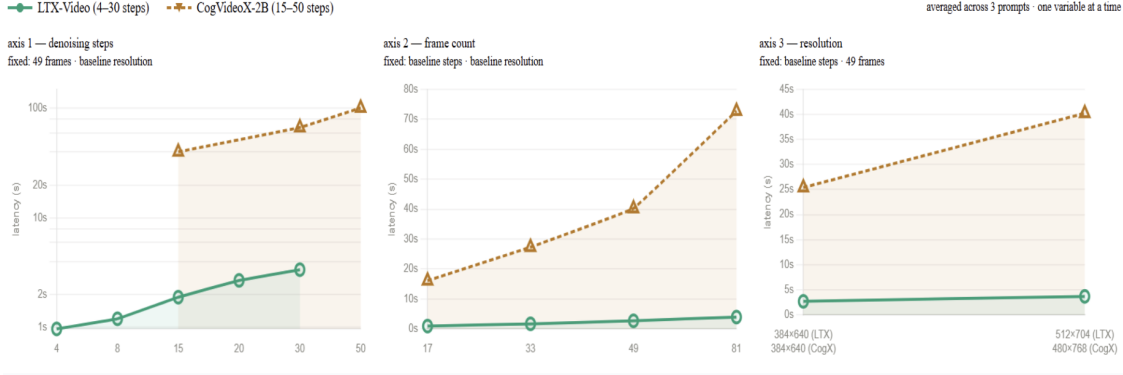


Figure 3: Increasing denoising steps, frame count, and resolution all increase latency, but the effect is substantially larger for CogVideoX than for LTX. The consistent pattern across all three sweeps suggests that denoising work is the dominant latency bottleneck.

ing loop. In other words, the latency difference is not simply an implementation artifact; it follows from how much repeated denoising work each model performs.

This is the main latency insight for the project. Once latency is part of the comparison, many CogVideoX-2B configurations that look attractive from quality alone become less competitive, while lower-latency LTX-Video configurations become more attractive under realistic budgets.

3.5 CPU Offload: Memory–Latency Tradeoff

Latency is not the only constraint. To isolate memory effects, we run a controlled CPU offload ablation on three matched CogVideoX-2B configurations. Within each matched pair, the prompt, model, denoising steps, frame count, resolution, and precision are held fixed; the only change is whether CPU offload is enabled.

Table 2: CPU offload ablation averaged across three matched CogVideoX-2B configurations.

Metric	Offload	No offload	Effect
Latency	22.1 s	16.5 s	+34%
GPU memory	11.1 GB	18.8 GB	−41%

CPU offload reveals a direct memory–latency tradeoff: it reduces peak GPU memory by 41%, but increases latency by 34%. In absolute terms,

memory drops from 18.8 GB to 11.1 GB, while latency rises from 16.5 s to 22.1 s. Offload therefore changes which CogVideoX-2B configurations are feasible under a memory budget, but it does so by adding runtime overhead. The optimal choice depends on which resource is scarce: a memory-constrained system may need offload, while a latency-constrained system favors no offload or a faster configuration.

3.6 Pareto Frontier: Which Configurations Are Competitive?

We next combine quality and latency directly. A configuration is dominated if another configuration achieves both higher quality and lower latency. Dominated configurations are never worth scheduling under the same objective.



Figure 4: Latency–quality tradeoff across 54 profiled configurations. 49 configurations are dominated once latency is considered.

The key result in Figure 4 is that 49 of 54 configurations are dominated. This directly answers our research question: configurations that look strong by quality alone can become noncompetitive once latency is included. In particular, many CogVideoX-2B configurations appear attractive if we only consider quality, but under realistic latency budgets many become dominated or infeasible. Much of the feasible region shifts toward lower-latency LTX-Video configurations. This demonstrates that resource constraints can change the conclusions drawn from quality-only rankings: a configuration that appears competitive in isolation may be eliminated once latency is included.

This pruning is useful because it turns the measured tradeoff curve into a smaller scheduling problem. The frontier reduces the candidate set from 54 configurations to 5 Pareto-optimal choices for the measured aggregate frontier. In prompt-specific decisions, the same principle applies: V-Scale only needs to compare configurations that are competitive for the given budget and prompt.

3.7 Scheduler vs. Fixed Presets

We now test whether the measured tradeoffs improve actual scheduling decisions. A fixed preset commits to a predefined configuration, while V-Scale adapts model choice and generation settings to the prompt and latency budget. We define fixed presets as predefined LTX-Video and CogVideoX-2B preview/quality configurations. A preset is included in the comparison only if it produces a valid output and satisfies the same latency budget as V-Scale.

Table 3: Representative scheduler decisions compared with fixed presets. Different prompts and budgets favor different resource allocations, such as more frames, higher resolution, or a stronger model.

Case	Fixed	V-Scale	Gain
Fast / 5s	LTX 49f	LTX 81f	+0.0645
Fast / 40s	CogX 17f	CogX 49f	+0.0560
Walk / 5s	LTX 49f	LTX hi-res 49f	+0.1175

Table 3 shows representative cases; the aggregate

result is computed over all 18 comparable fixed-preset decisions. Across these comparisons, V-Scale achieves 17 wins, 1 tie, and 0 losses, with a mean quality gain of +0.0549.

The important result is not only that V-Scale wins, but why it wins. The gains do not come from one universally better model or one universally better setting. Instead, the best allocation of the latency budget changes with both prompt content and available time.

For the fast-action prompt at a 5s budget, V-Scale selects an 81-frame LTX-Video configuration instead of the 49-frame fixed preset, improving the quality proxy from 0.5868 to 0.6513. This prompt contains rapid temporal change, so additional frames provide more temporal coverage of the action. The measured gain suggests that, for this high-motion case, spending the budget on more frames is useful.

The walking-person prompt shows a different pattern. Under the same 5s budget, V-Scale selects a higher-resolution 49-frame LTX-Video configuration, improving the quality proxy from 0.7340 to 0.8515. Here, the frame count stays fixed while spatial resolution increases. This suggests that, for this human-motion prompt, the scheduler’s best measured use of the budget is improving spatial detail and consistency rather than adding temporal coverage. This is consistent with the prompt requiring a stable person appearance, body shape, and background structure across frames.

The fast-action 40s case shows the role of the latency budget itself. When more time is available, V-Scale can afford a stronger CogVideoX-2B configuration, selecting 49 frames instead of the 17-frame fixed preset and improving the quality proxy from 0.7125 to 0.7685. This shows that higher-latency configurations are not always worse; they become useful when the budget is large enough to make them feasible.

These examples illustrate the main scheduling insight: different prompt characteristics benefit from different allocations of the same compute budget. High-motion prompts can benefit from temporal coverage, human-motion prompts can

benefit from spatial resolution, and larger budgets can make more expensive CogVideoX-2B configurations worthwhile. Therefore, no single fixed preset is optimal across all scenarios. By adapting model choice and generation settings to the available budget, V-Scale consistently finds higher-quality configurations than fixed presets.

Together, the Pareto analysis and fixed-preset comparison support both success criteria: resource constraints change which configurations are competitive, and budget-aware scheduling improves delivered quality over fixed presets under the same latency budgets.

3.8 Implementation Optimization Experiments

We also tested `torch.compile` [3] and INT8 quantization [4] as implementation-level optimization checks. The goal of this experiment is to test whether the large latency gap between LTX-Video and CogVideoX-2B is mainly an artifact of an unoptimized implementation, or whether it persists even after standard inference optimizations.

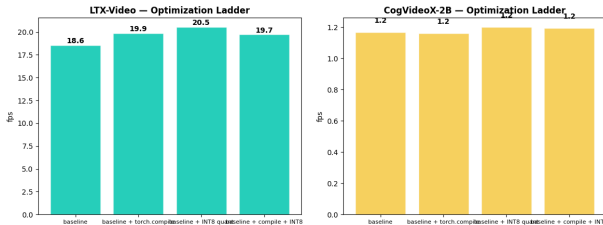


Figure 5: Throughput optimization results. `torch.compile` and INT8 improve absolute FPS, but the improvement is small relative to the model-level latency gap observed in the profiling sweeps.

Figure 5 shows that these optimizations improve absolute throughput, but only modestly. For LTX-Video, the combined optimization improves throughput from 18.9 FPS to 19.7 FPS, a $1.04\times$ speedup. For CogVideoX-2B, throughput improves from 1.1 FPS to 1.2 FPS, a $1.09\times$ speedup. These gains are useful, but they are much smaller than the cross-model throughput gap: LTX-Video is about $17.2\times$ faster at baseline and remains about $16.4\times$ faster after both optimizations.

This result is important because it rules out a simpler explanation of our earlier findings. The large latency difference is not primarily caused by missing implementation optimizations; it persists even after applying standard throughput improvements. Instead, the bottleneck analysis points to model-level structure: CogVideoX-2B performs substantially more repeated denoising work, so small kernel-level speedups do not fundamentally change the quality-latency frontier.

Therefore, implementation optimizations improve absolute throughput, but they do not remove the need for constraint-aware scheduling. The scheduler still needs to choose among models and configurations based on measured latency, memory, and quality tradeoffs.

3.9 Limitations and Failure Cases

The main limitation is the quality proxy. It is lightweight and useful for relative scheduling decisions, but it is not a substitute for a human perceptual study. The proxy can overvalue motion even when the motion is repetitive or physically implausible, and it may miss semantic errors that humans would notice. We mitigate this through validity checks and manual spot checks, but a larger human evaluation would be needed to validate fine-grained score differences.

A second limitation is hardware specificity. V-Scale profiles configurations once per hardware target; if the GPU or available memory changes, the measurements should be repeated. A natural extension would be to learn a latency prediction model that generalizes across hardware and interpolates between profiled configurations.

Finally, our prompt set is intentionally small. It covers low, medium, and high motion regimes, but broader prompt coverage would be needed before deploying this scheduler as a general-purpose video generation service.

4 Team Responsibilities

Jenny Jin. V-Scale pipeline design, CogVideoX bring-up, profiling/evaluation scripts, scheduler and fixed-preset comparison, Pareto analysis, bottleneck evaluation, and optimization analysis.

Jeffrey Liu. LTX-Video bring-up and validation, additional benchmarking experiments, preliminary bottleneck analysis, and optimization evaluation.

References

- [1] LTX-Video. <https://github.com/Lightricks/LTX-Video>
- [2] CogVideoX. <https://github.com/THUDM/CogVideo>
- [3] PyTorch `torch.compile` documentation. <https://pytorch.org/docs/stable/torch.compiler.html>
- [4] Bitsandbytes INT8 quantization. <https://github.com/TimDettmers/bitsandbytes>
- [5] Hugging Face Diffusers video generation documentation. <https://huggingface.co/docs/diffusers/v0.33.1/en/using-diffusers/text-img2vid>
- [6] Hugging Face Diffusers memory optimization documentation. <https://huggingface.co/docs/diffusers/en/optimization/memory>